

Lab 4

Designing a Pipelined RISC Processor (Part 1)

NAME: JABER-UL HUDA

Student ID: 101137524

I. **Introduction**

The objective of this lab is to design and implement the essential building blocks of a pipelined RISC (Reduced Instruction Set Computer) processor capable of executing 16 distinct instructions, including load, store, arithmetic, and logical operations. This processor will not support branch or control instructions. The lab focuses on developing and verifying the core functional units — instruction fetch, instruction decode, execution, and register file — using provided Verilog templates. Each component will be simulated and tested individually in ModelSim to ensure proper operation. Through this process, I will gain hands-on experience with RISC processor architecture, instruction formats, and ALU operations. In the following lab, these components will be integrated with instruction and data memory modules to complete the processor design and perform comprehensive system-level testing.

- II. **Design (Task 1-2):** Create one section for each design unit and include the corresponding Verilog code for the module definition and implementation.

The lab focused on designing and simulating the core components of a pipelined RISC processor featuring a 16-bit instruction set, an 8-register file, and a 4-stage pipeline architecture. The main modules included the instruction unit, decode unit, execution unit, and register file. The instruction unit was responsible for fetching instructions, the decode unit interpreted opcodes, operands, and addressing information, the execution unit implemented an ALU to perform arithmetic, logical, and shift operations, and the register file handled data storage and retrieval. Each unit was independently simulated and verified using ModelSim to ensure correct functionality and alignment with the processor's design objectives. For part 1 we are just defining the modules of the Instruction unit module, decode unit module, Execution unit module and Register file module. All the 4 modules are tested with the testbench code provided by the professor.

Instruction unit module:

```
18 //behavioral iunit
19 module risc_iunit (
20     // fillout1: complete the interface based on the given block diagram in the manual
21     input rst_n,
22     input clk,
23     input[12:0] instruction, //instruction from instrucion memory
24     output reg [4:0] pc, //pointer to next insruction in instruction memory
25     output reg [12:0] ir // 13 bit instruction to the decode stage
26 );
27
28     parameter nop = 13'h0000;
29
30     always @(posedge clk or negedge rst_n) begin
31         if (rst_n == 1'b0) begin
32             //fillout2: reset the ir to nop and reset the PC
33             ir <= nop;
34             pc <= 5'b00000;
35
36         end else begin
37             // fillout3:pass the input instruction to the output port and increement the PC
38             // if the PC reached maximum value you must reset it
39             ir <= instruction;
40             if (pc == 5'b11111)
41                 pc <= 5'b00000;
42
43             else
44                 pc <= pc + 5'b 00001;
45
46         end
47     end
48 endmodule
```

Decode unit module:

```
1  // NAME: JABER-UL HUDA
2  // ID: 101137524
3
4  module risc_decode (
5      input clk,
6      input rst_n,
7      // fillout 1: complete the following interface description
8      input [12:0] instr,
9      output reg [3:0] dmaddr,
10     output reg [3:0] opcode,
11     output reg [3:0] opnda,
12     output reg [3:0] opndb,
13     output reg [3:0] dst
14 );
15
16     // Internal signals
17     wire [3:0] opcode_i;
18     reg [3:0] dmaddr_i;
19     reg [2:0] opnda_i, opndb_i, dst_i;
20
21     parameter ld = 4'b1110, st = 4'b1111;
22
23     // fillout2: extract the opcode from the instruction and assign it to opcode_i
24     assign opcode_i = instr[12:9];
25
26     // in this always block you need to check the opcode,
27     // and depending on the opcode you will have to extract information accordingly
28     // note that the format for arithmetic/logical instructions are the same,
29     // so you need to check if the opcode represents a load, a store or arithmetic/logical
30     // make sure to consider a default for your case, so if something goes wrong it resets
31     // to a specific case
32     // refer to the instructions format given in the lab document
```

```

31 // to a specific case
32 // refer to the instructions format given in the lab document
33
34 always @(*) begin
35     case (opcode_i)
36         ld: begin
37             dmaddr_i = instr[7:4];
38             dst_i = instr[2:0];
39             opnda_i = 3'b000;
40             opndb_i = 3'b000;
41         end
42         st: begin
43             dst_i = instr[6:4];
44             dmaddr_i = instr[3:0];
45             opnda_i = 3'b000;
46             opndb_i = 3'b000;
47         end
48         4'b0001, 4'b0001, 4'b0010, 4'b0011, 4'b0100, 4'b0101, 4'b0110, 4'b0111, 4'b1000,
49         opnda_i = instr[8:6];
50         opndb_i = instr[5:3];
51         dst_i = instr[2:0];
52     end
53
54     default: begin
55         dst_i = 3'b000;
56         dmaddr_i = 4'b0000;
57         opnda_i = 3'b000;
58         opndb_i = 3'b000;
59     end
60 endcase
61 end
62

```

```

61     end
62
63     always @(posedge clk or negedge rst_n) begin
64         if (~rst_n) begin
65             // fillout4: reset all outputs with 0
66             dst <= 3'b000;
67             dmaddr <= 4'b0000;
68             opnda <= 3'b000;
69             opndb <= 3'b000;
70             opcode <= 4'b0000;
71         end else begin
72             // fillout5:
73             // pass the extracted information from the
74             // instructions to the next unit, this include opcode,
75             // dmaddr, dst, opnda, and opndb
76             dst <= dst_i;
77             dmaddr <= dmaddr_i;
78             opnda <= opnda_i;
79             opndb <= opndb_i;
80             opcode <= opcode_i;     end
81     end
82
83 endmodule
84

```

Execution unit module:

```
1  // NAME: JABER-UL HUDA
2  // ID: 101137524
3  |
4  module risc_eunit (
5      input clk,
6      input rst_n,
7      input [3:0] dmaddrin,
8      input [3:0] opcode,
9      input [7:0] oprnd_a,
10     input [7:0] oprnd_b,
11     input [2:0] dstin,
12     output reg dmenbl,
13     output wire rdwr,
14     output wire reg_wr_vld,
15     output wire load_op,
16     output reg [3:0] dmaddr,
17     output reg [7:0] rslt,
18     output wire [7:0] dmdatain,
19     output reg [2:0] dst
20 );
21
22 // Internal signals
23 wire [7:0] adder_in_a;
24 wire ci; // carry in
25 reg [7:0] rslt_i;
26 reg [7:0] rslt_sum;
27 reg [7:0] adder_in_b;
28 reg adder_mode;
29 reg co; // carry out
30 reg [3:0] opcode_out;
31
```

```

31
32     parameter ADD = 1'b0;
33     parameter SUB = 1'b1;
34     parameter NOP_OP = 4'b0000,
35           ADD_OP = 4'b0001,
36           SUB_OP = 4'b0010,
37           AND_OP = 4'b0011,
38           OR_OP = 4'b0100,
39           XOR_OP = 4'b0101,
40           INC_OP = 4'b0110,
41           DEC_OP = 4'b0111,
42           NOT_OP = 4'b1000,
43           NEG_OP = 4'b1001,
44           SHR_OP = 4'b1010,
45           SHL_OP = 4'b1011,
46           ROR_OP = 4'b1100,
47           ROL_OP = 4'b1101,
48           LD_OP = 4'b1110,
49           ST_OP = 4'b1111;
50
51     // Use assign statements to show mix of dataflow and behavioral
52     assign load_op = (opcode_out == LD_OP);
53     assign rdwr = (opcode_out == ST_OP) ? 1'b0 : 1'b1; // 0 for write (store), 1 for read (load)
54     assign reg_wr_vld = (opcode_out != ST_OP) && (opcode_out != NOP_OP);
55     assign dmdatoin = rslt;
56

```

```

56
57     always @(posedge clk or negedge rst_n) begin
58         if (~rst_n) begin
59             rslt <= 8'h00;
60             dmaddr <= 4'h0;
61             opcode_out <= 4'h0;
62             dst <= 3'b000;
63         end else begin
64             rslt <= rslt_i;
65             dst <= dstin;
66             dmaddr <= dmaddrin;
67             opcode_out <= opcode;
68         end
69     end
70
71     always @(posedge clk or negedge rst_n) begin
72         if (~rst_n)
73             dmenbl <= 1'b0;
74         else if (opcode == ST_0P || opcode == LD_0P)
75             dmenbl <= 1'b1;
76         else
77             dmenbl <= 1'b0;
78     end
79

```

```

79
80     always @(opcode) begin
81         // Fillout 1: if opcode is SHL_OP or DEC_OP set adder_mode to SUB otherwise set it to ADD
82         if (opcode == SHL_OP || opcode == DEC_OP) begin
83             adder_mode = SUB;
84         end else begin
85             adder_mode = ADD;
86         end
87     end
88
89     // Determine the b-input into the adder based on the opcode
90     always @(opcode or oprnd_b) begin
91         case (opcode)
92             SUB_OP: adder_in_b = ~oprnd_b; // Fillout 2: pass not of operand b
93             INC_OP: adder_in_b = 8'h01;
94             NEG_OP: adder_in_b = 8'h01;
95             DEC_OP: adder_in_b = ~(8'h01);
96             default: adder_in_b = oprnd_b;
97         endcase
98     end
99
100     assign adder_in_a = (opcode == NEG_OP) ? ~oprnd_a : oprnd_a;
101     assign ci = adder_mode;
102
103     always @(adder_in_a or adder_in_b or ci) begin
104         {co, rslt_sum} = adder_in_a + adder_in_b + ci;
105     end
106

```

```

106
107     // Mux in the result based on opcode
108     always @(opcode or oprnd_a or oprnd_b or rslt_sum) begin
109         case (opcode)
110             // Fillout 3: complete this section
111             ST_OP : rslt_i = oprnd_a;
112             NOP_OP: rslt_i = 8'h00;
113             AND_OP: rslt_i = oprnd_a & oprnd_b; // Calculate and of a and b
114             OR_OP : rslt_i = oprnd_a | oprnd_b; // Calculate or of a and b
115             XOR_OP: rslt_i = oprnd_a ^ oprnd_b;
116             SHR_OP: rslt_i = oprnd_a >> 1;
117             SHL_OP: rslt_i = oprnd_a << 1;
118             ROR_OP: rslt_i = {oprnd_a[0], oprnd_a[7:1]};
119             ROL_OP: rslt_i = {oprnd_a[6:0], oprnd_a[7]};
120             NOT_OP: rslt_i = ~oprnd_a; // Not of operand a
121             ADD_OP,
122             SUB_OP,
123             INC_OP,
124             DEC_OP,
125             NEG_OP: rslt_i = rslt_sum;
126             default: rslt_i = 8'h00;
127         endcase
128     end
129
130 endmodule
131

```

Register file module:

```
1  // NAME: JABER-UL HUDA
2  // ID: 101137524
3
4  module risc_regfile (
5      input clk,
6      input rst_n,
7      input reg_wr_vld,
8      input load_op,
9      input [7:0] rslt,
10     input [7:0] dmdataout,
11     input [2:0] dst,
12     input [2:0] opnda_addr,
13     input [2:0] opndb_addr,
14     output reg [7:0] oprnd_a,
15     output reg [7:0] oprnd_b
16 );
17
18     reg [7:0] regfile [0:7];
19     wire [7:0] reg_data_in;
20
21     /// fillout 1: if load_op is 1 assign dmdataout to reg_data_in otherwise rslt
22     assign reg_data_in = (load_op) ? dmdataout : rslt;
23
```

```

20
21 /// fillout 1: if load_op is 1 assign dmdataout to reg_data_in otherwise rslt
22 assign reg_data_in = (load_op) ? dmdataout : rslt;
23
24 always @(posedge clk or negedge rst_n) begin
25     if (~rst_n) begin
26         regfile[0] <= 8'h00;
27         regfile[1] <= 8'h22;
28         regfile[2] <= 8'h44;
29         regfile[3] <= 8'h66;
30         regfile[4] <= 8'h88;
31         regfile[5] <= 8'haa;
32         regfile[6] <= 8'hcc;
33         regfile[7] <= 8'hff;
34     end else if (reg_wr_vld) begin
35         case (dst)
36             // fillout2
37             // this is for load operation,
38             // depending on dst we write the data in the
39             // destination register specified by dst
40             3'd0: regfile[0] <= reg_data_in;
41             3'd1: regfile[1] <= reg_data_in;
42             3'd2: regfile[2] <= reg_data_in;
43             3'd3: regfile[3] <= reg_data_in;
44             3'd4: regfile[4] <= reg_data_in;
45             3'd5: regfile[5] <= reg_data_in;
46             3'd6: regfile[6] <= reg_data_in;
47             3'd7: regfile[7] <= reg_data_in;
48         endcase
49     end
50 end

```

```

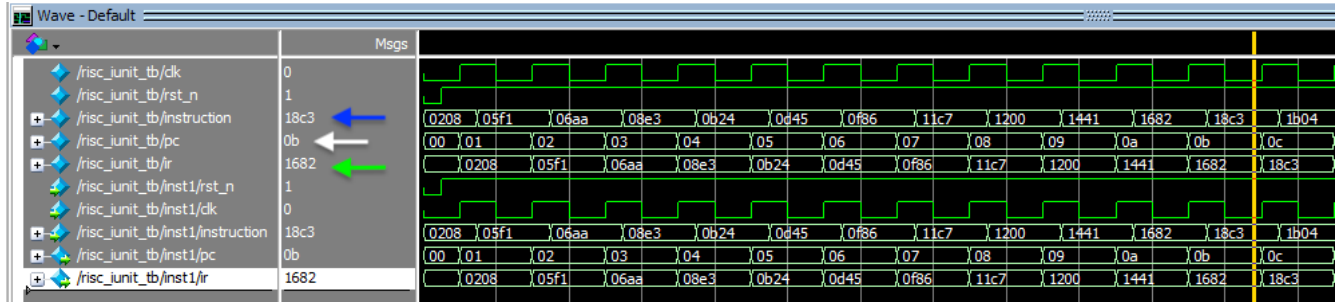
51
52     always @(*) begin
53         case (opnda_addr)
54             // fillout 3: pass appropriate register to oprnd_a based on opnd_addr
55             //
56             3'd0: oprnd_a = regfile[0];
57             3'd1: oprnd_a = regfile[1];
58             3'd2: oprnd_a = regfile[2];
59             3'd3: oprnd_a = regfile[3];
60             3'd4: oprnd_a = regfile[4];
61             3'd5: oprnd_a = regfile[5];
62             3'd6: oprnd_a = regfile[6];
63             3'd7: oprnd_a = regfile[7];
64             default: oprnd_a = 8'h00;
65         endcase
66     end
67
68     always @(*) begin
69         // fillout 4: pass appropriate register to oprnd_b based on opnd_addr
70         //
71         case (opndb_addr)
72             3'd0: oprnd_b = regfile[0];
73             3'd1: oprnd_b = regfile[1];
74             3'd2: oprnd_b = regfile[2];
75             3'd3: oprnd_b = regfile[3];
76             3'd4: oprnd_b = regfile[4];
77             3'd5: oprnd_b = regfile[5];
78             3'd6: oprnd_b = regfile[6];
79             3'd7: oprnd_b = regfile[7];
80             default: oprnd_b = 8'h00;
81         endcase
82     end
83 endmodule

```

- III. **Test (Task 3):** Test the design for each operation/scenario, take one screenshot of each design unit, mark the screenshot for better understanding, analyze and explain the performance (at least two screenshots for each unit: one for the waveform and one for the transcript).

Instruction unit module:

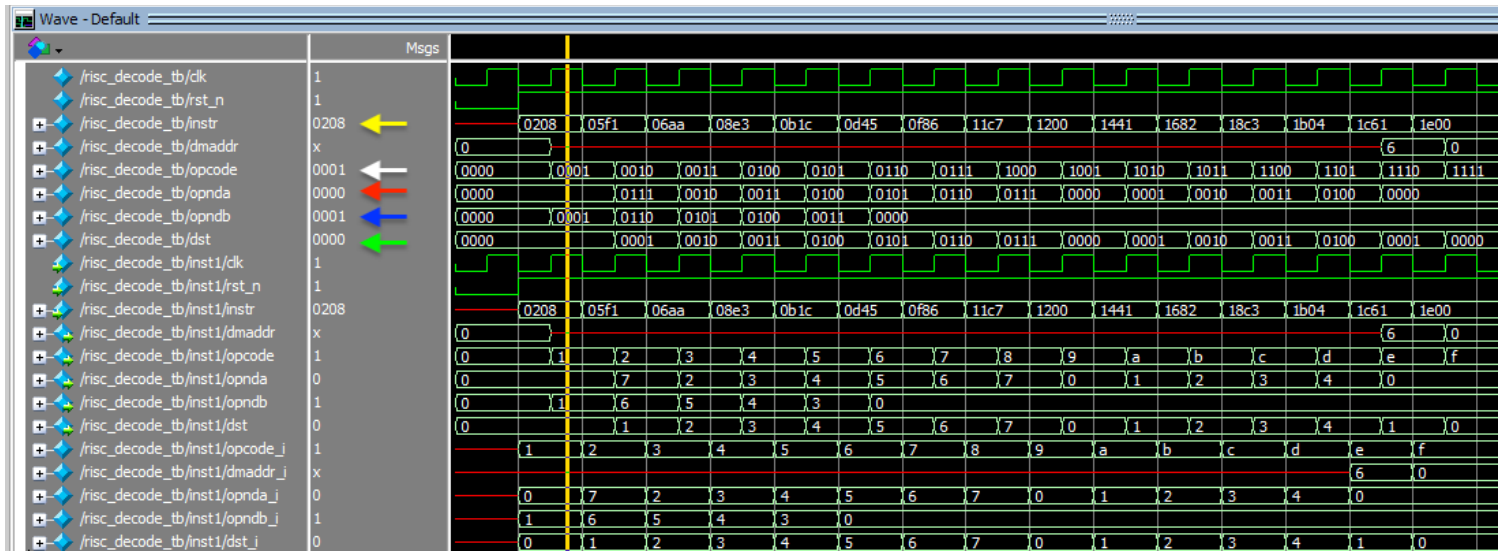
The white arrow shows the pc getting counted-up from 0x0 to 0xC and the blue arrow show the instruction getting inputted and the green arrow shows the same instruction pushed out as ir.



```
# pc = xx, instruction = 0208, ir = xxxx
# pc = 01, instruction = 05f1, ir = 0208
# pc = 02, instruction = 06aa, ir = 05f1
# pc = 03, instruction = 08e3, ir = 06aa
# pc = 04, instruction = 0b24, ir = 08e3
# pc = 05, instruction = 0d45, ir = 0b24
# pc = 06, instruction = 0f86, ir = 0d45
# pc = 07, instruction = 11c7, ir = 0f86
# pc = 08, instruction = 1200, ir = 11c7
# pc = 09, instruction = 1441, ir = 1200
# pc = 0a, instruction = 1682, ir = 1441
# pc = 0b, instruction = 18c3, ir = 1682
# pc = 0c, instruction = 1b04, ir = 18c3
```

Decode unit module:

The yellow arrow shows the instruction inputted here it is 0x0208. The white arrow shows the opcode form the instruction 0x0208. The red arrow shows the opnda and the blue arrow show the opndb both are decoded from the instruction 0x0208. The green arrow shows the dst address of the result form the ALU.

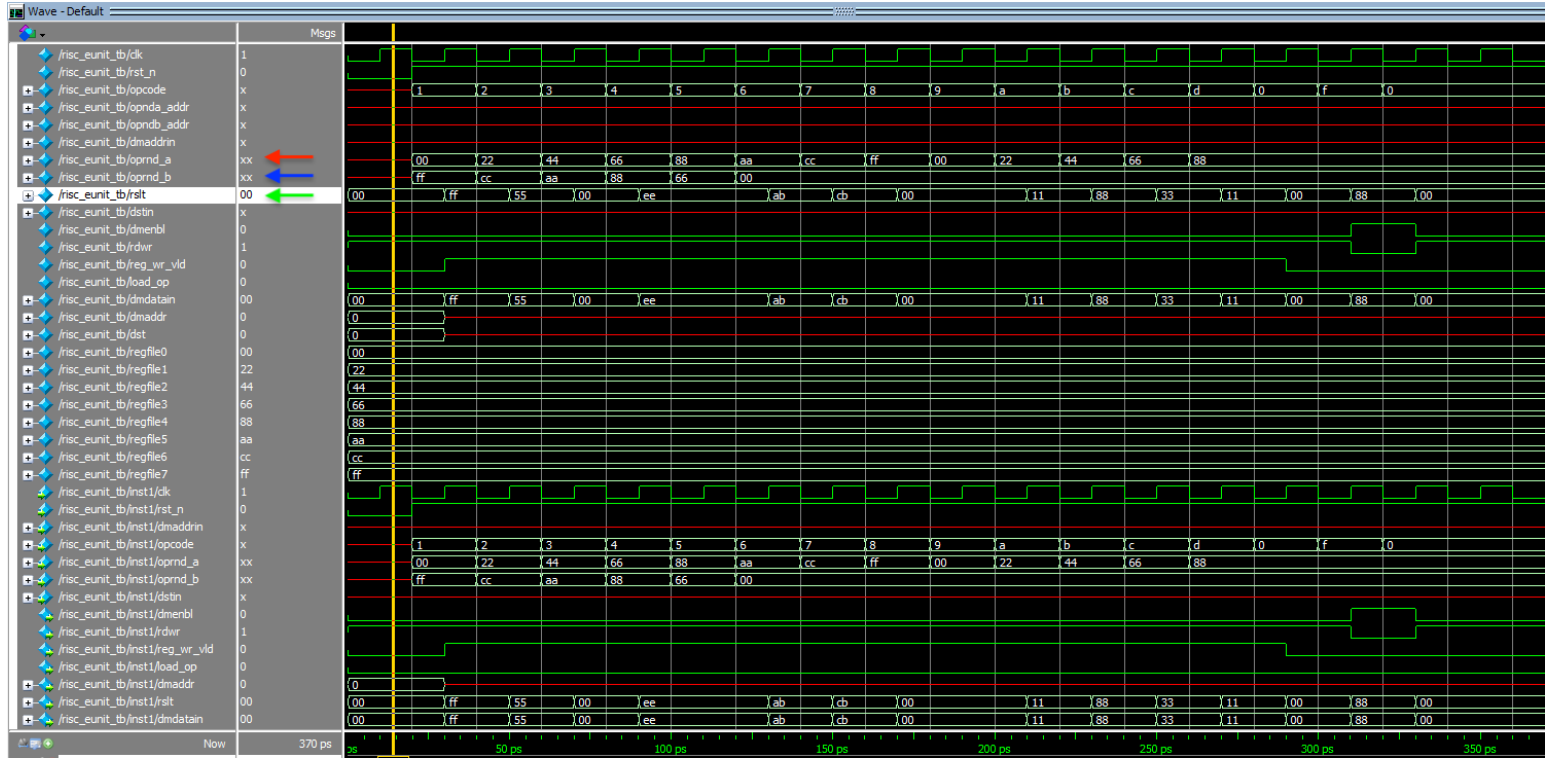


```
VSIM 19> run -all
```

```
# opcode=0001, opnda=0000, opndb=0000, dst=0000
# opcode=0010, opnda=0000, opndb=0001, dst=0000
# opcode=0011, opnda=0111, opndb=0110, dst=0001
# opcode=0100, opnda=0010, opndb=0101, dst=0010
# opcode=0101, opnda=0011, opndb=0100, dst=0011
# opcode=0110, opnda=0100, opndb=0011, dst=0100
# opcode=0111, opnda=0101, opndb=0000, dst=0101
# opcode=1000, opnda=0110, opndb=0000, dst=0110
# opcode=1001, opnda=0111, opndb=0000, dst=0111
# opcode=1010, opnda=0000, opndb=0000, dst=0000
# opcode=1011, opnda=0001, opndb=0000, dst=0001
# opcode=1100, opnda=0010, opndb=0000, dst=0010
# opcode=1101, opnda=0011, opndb=0000, dst=0011
# opcode=1110, opnda=0100, opndb=0000, dst=0100
# opcode=1111, opnda=0000, opndb=0000, dst=0001
```

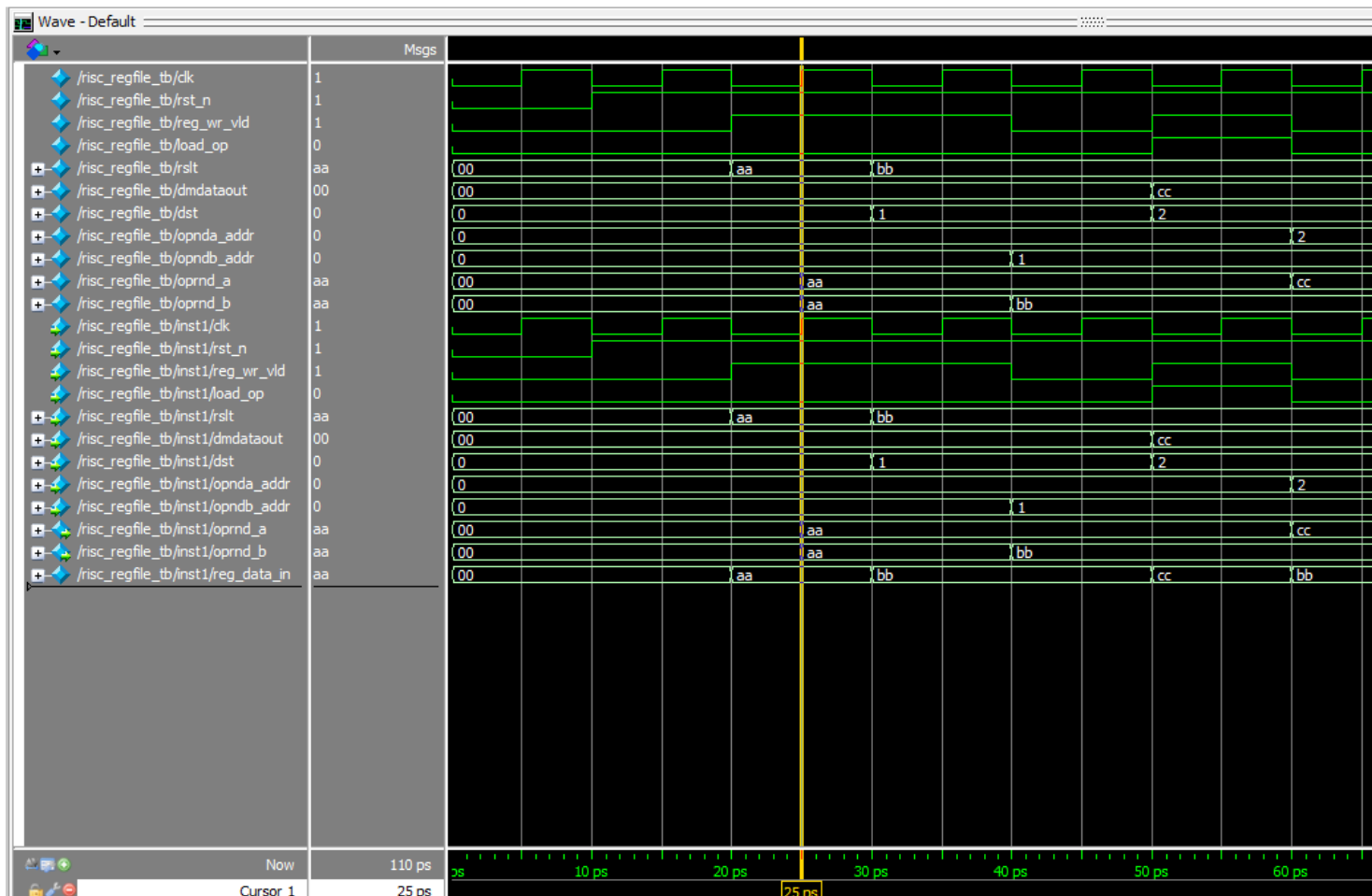
Execution unit module:

The line above the red arrow is the opcode indicating what to do. The red arrow shows the oprnd_a and the blue arrow shows the oprnd_b these 2 are the inputs of the ALU. The green arrow is the output of the ALU.



```
# regfile0 = 00, regfile7 = ff, add, rslt = ff
# regfile1 = 22, regfile6 = cc, sub, rslt = 55
# regfile2 = 44, regfile5 = aa, and, rslt = 00
# regfile3 = 66, regfile4 = 88, or , rslt = ee
# regfile4 = 88, regfile3 = 66, xor, rslt = ee
# regfile5 = aa, regfile0 = 00, inc, rslt = ab
# regfile6 = cc, regfile0 = 00, dec, rslt = cb
# regfile7 = ff, regfile0 = 00, not, rslt = 00
# regfile0 = 00, regfile0 = 00, neg, rslt = 00
# regfile1 = 22, regfile0 = 00, shr, rslt = 11
# regfile2 = 44, regfile0 = 00, shl, rslt = 88
# regfile3 = 66, regfile0 = 00, ror, rslt = 33
# regfile4 = 88, regfile0 = 00, rol, rslt = 11
```

Register file module:



```
# Time=0 clk=0 rst_n=0 reg_wr_vld=0 load_op=0 rslt=00 dmdataout=00 dst=0 opnda_addr=0 opndb_addr=0 oprnd_a=00 oprnd_b=00
# Time=5 clk=1 rst_n=0 reg_wr_vld=0 load_op=0 rslt=00 dmdataout=00 dst=0 opnda_addr=0 opndb_addr=0 oprnd_a=00 oprnd_b=00
# Time=10 clk=0 rst_n=1 reg_wr_vld=0 load_op=0 rslt=00 dmdataout=00 dst=0 opnda_addr=0 opndb_addr=0 oprnd_a=00 oprnd_b=00
# Time=15 clk=1 rst_n=1 reg_wr_vld=0 load_op=0 rslt=00 dmdataout=00 dst=0 opnda_addr=0 opndb_addr=0 oprnd_a=00 oprnd_b=00
# Time=20 clk=0 rst_n=1 reg_wr_vld=1 load_op=0 rslt=aa dmdataout=00 dst=0 opnda_addr=0 opndb_addr=0 oprnd_a=00 oprnd_b=00
# Time=25 clk=1 rst_n=1 reg_wr_vld=1 load_op=0 rslt=aa dmdataout=00 dst=0 opnda_addr=0 opndb_addr=0 oprnd_a=aa oprnd_b=aa
# Time=30 clk=0 rst_n=1 reg_wr_vld=1 load_op=0 rslt=bb dmdataout=00 dst=1 opnda_addr=0 opndb_addr=0 oprnd_a=aa oprnd_b=aa
# Time=35 clk=1 rst_n=1 reg_wr_vld=1 load_op=0 rslt=bb dmdataout=00 dst=1 opnda_addr=0 opndb_addr=0 oprnd_a=aa oprnd_b=aa
# Time=40 clk=0 rst_n=1 reg_wr_vld=0 load_op=0 rslt=bb dmdataout=00 dst=1 opnda_addr=0 opndb_addr=1 oprnd_a=aa oprnd_b=bb
# Time=45 clk=1 rst_n=1 reg_wr_vld=0 load_op=0 rslt=bb dmdataout=00 dst=1 opnda_addr=0 opndb_addr=1 oprnd_a=aa oprnd_b=bb
# Time=50 clk=0 rst_n=1 reg_wr_vld=1 load_op=1 rslt=bb dmdataout=cc dst=2 opnda_addr=0 opndb_addr=1 oprnd_a=aa oprnd_b=bb
# Time=55 clk=1 rst_n=1 reg_wr_vld=1 load_op=1 rslt=bb dmdataout=cc dst=2 opnda_addr=0 opndb_addr=1 oprnd_a=aa oprnd_b=bb
# Time=60 clk=0 rst_n=1 reg_wr_vld=0 load_op=0 rslt=bb dmdataout=cc dst=2 opnda_addr=2 opndb_addr=1 oprnd_a=cc oprnd_b=bb
# Time=65 clk=1 rst_n=1 reg_wr_vld=0 load_op=0 rslt=bb dmdataout=cc dst=2 opnda_addr=2 opndb_addr=1 oprnd_a=cc oprnd_b=bb
# Time=70 clk=0 rst_n=1 reg_wr_vld=0 load_op=0 rslt=bb dmdataout=cc dst=2 opnda_addr=2 opndb_addr=1 oprnd_a=cc oprnd_b=bb
```

IV. Conclusion

In this lab, I gained valuable hands-on experience in designing and simulating the fundamental components of a pipelined RISC processor. By implementing the instruction unit, decode unit, execution unit, and register file, I developed a stronger understanding of how individual modules interact within a processor pipeline. Using ModelSim to test each component helped reinforce my knowledge of instruction formats, ALU operations, and data flow within a RISC architecture. One of the key challenges was ensuring that each module functioned correctly and produced accurate outputs during simulation, which required careful debugging and signal tracing. Overcoming these challenges deepened my understanding of modular design and verification in hardware systems. Overall, this experiment enhanced my comprehension of processor architecture and prepared me for the next stage—integrating these components into a complete pipelined processor with memory modules for full system testing.

V. Copy the rubric (given on the next pages of this lab manual) and include it after the conclusion section in your lab report so that your teaching assistant can mark your report accordingly.

Criteria	Level 4	Level 3	Level 2	Level 1	Criterion Score
Register file unit code and simulation	20 points The register file code is complete and correct and simulation screenshots are included and explained	16 points Simulation is not complete and may have minor errors or is not explained	10 points The code is not complete, simulation is not included, and/or has major issues	0 points this block is not completed.	/ 20
instruction fetch unit	20 points The unit is complete, correct and simulation screenshots are included and explained	17 points Simulation is not complete and may have minor errors or is not explained	12 points The code is not complete, simulation is not included, or has major issues	0 points this block is not completed.	/ 20
execution unit	20 points The unit is complete, correct and simulation screenshots are included and explained	17 points Simulation is not complete and may have minor errors or is not explained	12 points The code is not complete, simulation is not included, or has major issues	0 points this block is not completed.	/ 20
instruction decode	20 points The unit is complete, correct and simulation screenshots are included and explained	17 points Simulation is not complete and may have minor errors or is not explained	12 points The code is not complete, simulation is not included, or has major issues	0 points this block is not completed.	/ 20
Lab report and documentation	10 points All required screenshots are included and marked when necessary. The work is explained. Student name and ID are added to all codes. All codes are submitted as a separate file.	7 points Lab report has minor issues (more explanation is needed, screenshots are not visible enough, organization is not that good) or student name and id are missing or submission format is not right.	4 points Lab report misses key criteria outlined in level 4. The work is not explained well.	0 points The lab report does not meet the requirements. Most screenshots are missing, no explanations, etc.	/ 10
Checking out from the lab and answering questions	10 points The student attended the lab, checked out and explained his work thoroughly.	7 points The student could not answer some questions	4 points The student could not explain most of their work	0 points The student could not explain their work at all.	/ 10